MiniBankOS — Presentation Guide (5 Members)

Run Before ANY Demo

```
Remove-Item src\data\*.csv -Force
Remove-Item src\data\*.log -Force -ErrorAction SilentlyContinue
java -cp out Main
```

Then in the app: `login root root123`

MEMBER 1 — Synchronization: Reader-Writer Problem + Critical Section

What to Say to Mam

"In an OS, multiple processes may access shared memory simultaneously. Without synchronization, two threads writing at the same time corrupt data.

*Our project implements the **Reader-Writer Problem**: Multiple users can CHECK balance at the same time (shared read). But only ONE user can DEPOSIT or WITHDRAW at a time (exclusive write).*

*We use Java's ReentrantReadWriteLock — one lock per bank account. Each account is a **critical section** — code that only one thread enters at a time. This matches exactly what OS theory defines as the critical section problem."*

Code to Show — ReadWriteManager.java

```
// One lock per account = one lock per shared resource
private ConcurrentHashMap<String, ReentrantReadWriteLock> locks
    = new ConcurrentHashMap<>();

public ReentrantReadWriteLock getLock(String accountName) {
    locks.putIfAbsent(accountName, new ReentrantReadWriteLock());
    return locks.get(accountName);
}
```

Code to Show — BankDatabase.java (Read + Write Lock)

```
// READER: multiple threads can read balance simultaneously (shared)
lock.readLock().lock();
try {
    System.out.println("Balance = " + acc.getBalance());
} finally {
    lock.readLock().unlock();
}

// WRITER: only ONE thread can deposit at a time (critical section)
lock.writeLock().lock();
try {
    account.deposit(amount);      // exclusive access
    persistAccount(account);
} finally {
    lock.writeLock().unlock();
}
```

Demo Commands

```
create alice alice123 5000
create bob bob123 3000
balance alice
```

Say: "balance uses READ lock — multiple users can check simultaneously"

```
grant alice
logout
login alice alice123
deposit alice 1000
balance alice
```

Say: "deposit uses WRITE lock — only ONE thread inside critical section. Others WAIT outside. New balance 6000 — no corruption."

Viva Q: What is critical section?
A: "Code where shared resource is accessed — only one process allowed at a time. We use write lock as critical section for deposit/withdraw."

MEMBER 2 — Scheduling: Round Robin + Priority

What to Say to Mam

"An OS scheduler decides WHICH process runs NEXT and for HOW LONG.

Our project implements TWO scheduling algorithms simultaneously:

Round Robin: *Each transaction gets a fixed time slice (quantum = 3 units). After its time, it goes back to queue. Fair for all.*

Priority Scheduling: *High priority transactions (transfers) run before low priority ones (balance checks).*

Every banking operation becomes a ***TransactionProcess*** *— with a process ID, priority, and task — exactly like a PCB in OS."*

Code to Show — TransactionProcess.java

```
// Each operation becomes a process (like PCB in OS)
public class TransactionProcess {
    private String processId;   // unique process ID (like PID)
    private Runnable task;      // what the process does
    private int priority;       // higher = runs first
    private int timeSlice = 3;  // quantum for round robin
}
```

Code to Show — RoundRobinQueue.java

```
// FIFO queue = Round Robin base
private Queue<TransactionProcess> queue = new LinkedList<>();

public void addProcess(TransactionProcess p) {
    queue.add(p);        // add to tail (newest)
}
public TransactionProcess getNext() {
    return queue.poll(); // remove from head = FIFO = Round Robin
}
```

Priority Assignment — TransactionManager.java

```
// Transfer = priority 1 (high — two accounts involved)
new TransactionProcess("TX-"+id, task, 1);

// Create account = priority 2 (highest — setup operation)
new TransactionProcess("TX-"+id, task, 2);
```

Demo Commands

```
create alice alice123 5000
create bob bob123 3000
grant alice
logout
login alice alice123
deposit alice 100
```

Say: "deposit becomes a TransactionProcess, submitted to scheduler, assigned priority, runs with quantum."

```
transfer alice bob 500
```

Say: "transfer is higher priority — if both deposit and transfer are pending, transfer runs first."

Viva Q: What is time quantum in your project?
A: "Each TransactionProcess has timeSlice=3. The scheduler gives each process 3 execution units before moving to next — this is Round Robin quantum."

MEMBER 3 — Deadlock Prevention (Lock Ordering)

What to Say to Mam

"Deadlock occurs when two processes wait for each other forever.
Classic case: Thread 1 holds lock A, wants lock B. Thread 2 holds lock B, wants lock A. Both stuck.

Deadlock has 4 necessary conditions:
1. Mutual Exclusion 2. Hold and Wait 3. No Preemption 4. Circular Wait

If we eliminate even ONE — deadlock cannot occur.

*Our project eliminates **Circular Wait** by always locking accounts in alphabetical order, regardless of transfer direction. This is **Resource Ordering** — standard deadlock prevention."*

Code to Show — BankDatabase.java (transfer method)

```
// DEADLOCK PREVENTION: always lock in alphabetical order
ReentrantReadWriteLock lock1 = lockManager.getLock(from);
ReentrantReadWriteLock lock2 = lockManager.getLock(to);

ReentrantReadWriteLock firstLock  = from.compareTo(to) <= 0 ? lock1 : lock2;
ReentrantReadWriteLock secondLock = from.compareTo(to) <= 0 ? lock2 : lock1;

firstLock.writeLock().lock();   // lock alphabetically first account
secondLock.writeLock().lock();  // then second account
```

Why This Works

```
WITHOUT prevention:
  Thread 1: transfer alice→bob  → locks alice, then WAITS for bob
  Thread 2: transfer bob→alice  → locks bob,   then WAITS for alice
  Result: DEADLOCK (circular wait)

WITH prevention (alphabetical order):
  Thread 1: transfer alice→bob  → locks ALICE first, then BOB
  Thread 2: transfer bob→alice  → ALSO locks ALICE first (not bob!)
  Result: Thread 2 waits for alice → T1 finishes → releases both
          T2 then gets alice+bob → completes. No deadlock!
```

Demo Commands

```
create alice alice123 5000
create bob bob123 3000
grant alice
grant bob
logout
login alice alice123
transfer alice bob 1000
```

Say: "alice→bob: locks ALICE first (a < b alphabetically), then BOB"

```
balance alice
balance bob
```

Say: "alice=4000, bob=4000. Both correct — no deadlock, no corruption. Prevention works by design, no runtime check needed."

Viva Q: Difference between prevention and avoidance?
A: "Prevention: design-time rule, eliminates a deadlock condition permanently, zero runtime cost. Avoidance: runtime check before every resource grant, can block valid requests."

MEMBER 4 — Deadlock Avoidance (Banker's Algorithm)

What to Say to Mam

"Deadlock Avoidance checks at RUNTIME whether granting a resource is safe.

*We implemented **Banker's Algorithm** — Dijkstra named it after banks because a bank must keep enough reserve to serve all customers.*

In our project:
Resource = bank account (each account is a lockable resource)
Process = transfer transaction (needs 2 accounts: sender + receiver)

Before EVERY transfer, Safety Algorithm runs:
Step 1: Pretend to grant accounts to this process
Step 2: Find a safe sequence — order where ALL processes can finish
Step 3: Safe sequence found → ALLOW transfer
No safe sequence → BLOCK transfer (deadlock risk)"

Code to Show — BankersAlgorithm.java

```
public synchronized boolean requestAndCheck(String processId, Set<String> needed) {

    // Step 1: Tentatively allocate
    tempAlloc.put(processId, needed);
    tempAvailable.removeAll(needed);

    // Step 2: Safety Algorithm
    while (progress) {
        for each process:
            need = max - allocation        // what it still needs
            if (available >= need):        // can it finish with available?
                available += allocation    // release its resources
                safeSequence.add(process)
                progress = true
    }

    // Step 3: Decision
    if (all processes in safeSequence):
        return true;  // SAFE → allow transfer
    else:
        return false; // UNSAFE → block transfer
}
```

Integration — TransactionManager.java

```
// Before EVERY transfer — Banker's check runs
Set<String> needed = new HashSet<>(Arrays.asList(from, to));
if (!bankersAlgorithm.requestAndCheck(processId, needed)) {
    System.out.println("Transfer blocked — unsafe state.");
    return; // do NOT proceed
}
// Only reaches here if SAFE
submitKernelProcess(process);
bankersAlgorithm.release(processId); // release after done
```

Demo Commands

```
create alice alice123 5000
create bob bob123 3000
grant alice
logout
login alice alice123
transfer alice bob 1000
```

Output mam will see:

```
+-------------------------------------------------+
|       BANKER'S ALGORITHM  - SAFETY CHECK         |
+-------------------------------------------------+
Process ID          : TX-xxxxx
Requesting Accounts : [alice, bob]
All Resources       : [alice, bob]
Available Accounts  : [alice, bob]
Active Processes    : none
Safe Sequence       : [TX-xxxxx]
Result              : [SAFE] Transfer approved.

1000.0 transferred from alice to bob
[BANKER'S] Process TX-xxxxx released accounts: [alice, bob]
```

Say: "Safe sequence found — algorithm confirms all processes can complete. Resources granted. After transfer, resources released. System always in safe state."

Viva Q: What is safe state?
A: "A state where there exists at least one sequence in which all processes can complete without deadlock. Banker's ensures system never leaves safe state."

MEMBER 5 — Memory Management (Paging + LRU Page Replacement)

What to Say to Mam

"Memory management in OS involves two key concepts:

1. Paging with Address Translation:
Programs use logical names. OS converts them to physical locations via Page Table. Our PageTable maps operation names to CSV files — logical to physical address translation.

2. LRU Page Replacement:
Physical RAM is limited — holds only a few pages at a time. When RAM is full and a new page is needed, OS evicts the Least Recently Used page to disk.

In our project:
RAM (limited) = LinkedHashMap with MAX_FRAMES = 5 accounts
Disk = CSV files (all accounts stored permanently)
Page Fault = account not in cache, loaded from CSV"

Code to Show — PageTable.java (Address Translation)

```
// Logical operation name → Physical CSV file (address translation)
table.put("deposit",    "user.csv");
table.put("login",      "auth.csv");
table.put("loan-house", "house_loans.csv");

public String getFileName(String operation) {
    return table.get(operation);  // "deposit" → "user.csv"
}
// Program says "deposit" — PageTable resolves to "user.csv"
// Just like: virtual address → MMU → physical address
```

Code to Show — BankDatabase.java (LRU + Page Fault)

```
private static final int MAX_FRAMES = 5; // fixed RAM limit

// LRU cache — accessOrder=true means every get() moves entry to MRU
private final LinkedHashMap<String, Account> frames =
    new LinkedHashMap<String, Account>(MAX_FRAMES, 0.75f, true) {

    protected boolean removeEldestEntry(Map.Entry<String, Account> eldest) {
        if (size() > MAX_FRAMES) {
            System.out.println("[PAGE-OUT] Evicting '"
                + eldest.getKey() + "' (LRU)");
            return true; // evict LRU account to disk (CSV)
        }
        return false;
    }
};

// Page Fault Handler
private Account pageIn(String name) {
    System.out.println("[PAGE FAULT] '" + name + "' loading from disk...");
    Account account = readFromCSV(name); // read from disk
    frames.put(name, account);           // load into frame (may evict)
    System.out.println("[PAGE-IN] '" + name + "' loaded.");
    return account;
}
```

Demo Commands — Follow in Order

```
create p1 pass1 1000
```

Say: "Frame 1/5 used. [p1][EMPTY][EMPTY][EMPTY][EMPTY]"

```
create p2 pass2 2000
create p3 pass3 3000
create p4 pass4 4000
create p5 pass5 5000
```

Say: "Frames FULL. LRU order: [p1][p2][p3][p4][p5] — p1 is LRU (inserted first, never accessed)"

```
balance p1
```

Say: "TLB HIT — p1 accessed, moves to MRU end! Now p2 is LRU. Frame order: [p2][p3][p4][p5][p1]"

```
create p6 pass6 6000
```

Say: "6th account — frame full! LRU evicts p2 (NOT p1 — because p1 was just accessed). This is LRU vs FIFO difference!"

Output: `[PAGE-OUT] Evicting 'p2' (LRU)`

```
balance p2
```

Say: "p2 was evicted to disk. Accessing it = PAGE FAULT. OS loads p2 back from CSV into frame, evicts p3 (new LRU)."

Output: `[PAGE FAULT] 'p2' not in frames → loading from disk...`
`[PAGE-OUT] Evicting 'p3' (LRU)`
`[PAGE-IN] 'p2' loaded into frames.`

Viva Q: Why LRU and not FIFO?
A: "LRU evicts the account used least recently. If alice was just accessed, she'll likely be accessed again soon — LRU keeps her. FIFO ignores usage and evicts whoever arrived first. LRU gives better cache hit rate for banking workloads."

Quick Reference — All 5 Members

Member	Topic	Key File	Key Output
1	Reader-Writer + Critical Section	ReadWriteManager.java	Read/Write lock behavior on balance/deposit
2	Round Robin + Priority Scheduling	Scheduler.java, RoundRobinQueue.java	Every op becomes TransactionProcess with priority
3	Deadlock Prevention	BankDatabase.java transfer()	Alphabetical lock order → circular wait impossible
4	Deadlock Avoidance (Banker's)	BankersAlgorithm.java	Safety check + safe sequence printed before transfer
5	Memory Management (Paging + LRU)	PageTable.java, BankDatabase.java	Frame state + PAGE-OUT + PAGE FAULT + PAGE-IN

Common Viva Questions + Answers

Q: Is this a real OS or simulation?
A: "It is a software-level OS simulation in Java. JVM manages actual hardware. We implement OS algorithms at the application layer — standard for university OS projects."

Q: What is the difference between prevention and avoidance?
A: "Prevention eliminates a deadlock condition by design — lock ordering removes circular wait permanently. Avoidance checks safety at runtime before every resource grant using Banker's algorithm."

Q: Why LRU and not FIFO for page replacement?
A: "LRU evicts accounts unused the longest — exploits temporal locality. FIFO evicts the oldest arrival regardless of usage. LRU is more practical for banking where recently accessed accounts are likely to be accessed again."

Q: What is safe sequence in Banker's?
A: "The order in which all active processes can complete without deadlock. If no such sequence exists, the state is unsafe and we block the request."

Q: What is a page fault in your project?
A: "When an account is accessed but not in the LRU cache (frames), it triggers a page fault. The system reads the account from CSV (disk) and loads it into a frame — exactly like OS loading a page from swap space into RAM."